

Relazione Progettuale:

Progetto “Fast Food” — Programmazione Web e Mobile 2025/2026

1. Obiettivo del Progetto:

Il Progetto prevede l’implementazione di una piattaforma web completa per la gestione e la fruizione di un servizio “Fast Food” con due ruoli principali:

- **Cliente:** Può registrarsi, autenticarsi, sfogliare i menù dei ristoranti, aggiungere prodotti al carrello, effettuare ordini e consultarne lo storico.
- **Ristoratore:** Può creare/gestire il proprio ristorante, configurare il menù, monitorare ordini e statistiche, gestire la bacheca/offerte e contenuti collegati.

1.0 Architettura funzionale dei 4 macro-scenari:

Il Progetto si pone l’obiettivo di sviluppare l’applicazione web **Fast Food**, cioè un sito di ordinazione online per ristoranti appartenenti a una catena di Fast Food. In termini operativi, la piattaforma non si limita alla semplice selezione dei piatti, ma governa l’intero ciclo: accesso utente, gestione del ristorante, processamento ordini e supporto alla consegna.

Scenario (A) - Gestione del Profilo Utente:

Il primo macro-scenario (Gestione del profilo utente) riguarda la gestione classica dell’identità digitale dell’utente dentro Fast Food, dalla creazione dell’account fino alla possibile rimozione del profilo.

In termini funzionali, la piattaforma acquisisce e governa i dati principali dell’utente (anagrafica, credenziali e contatti), ne permette l’aggiornamento nel tempo e ne consente la cancellazione quando previsto dal flusso applicativo. Inoltre, nel progetto esistono due tipologie di utenza, **Cliente** e **Ristoratore**, che fanno parte di una scelta centrale poiché determinano sia la navigazione frontend sia i permessi backend.

La fase di registrazione deve quindi raccogliere almeno: **nome utente**, **indirizzo email**, **password** e soprattutto la **tipologia di utenza** (Ristoratore o Cliente). Questa classificazione iniziale condiziona l’intero percorso successivo, infatti, il Cliente accederà ai flussi di esplorazione menù, carrello e ordini, mentre il Ristoratore accederà ai flussi di creazione/gestione ristorante, menù e lavorazione ordini.

Operativamente, questo scenario copre tutto il ciclo di vita dell’account:

- **registrazione e autenticazione** (pagine **register.html**, **login.html**, **logout.html** e route **users.js**);
- **definizione della tipologia utente** (cliente/ristoratore), che abilita percorsi diversi dell’interfaccia e delle API;
- **aggiornamento dei dati personali** tramite area profilo (**profilo.html**) e relativi endpoint protetti;
- **rimozione del profilo** secondo le operazioni esposte dal dominio utenti.

“Relazione con gli altri Scenari”: senza identità utente non è possibile né aprire un ristorante (Scenario B), né creare/monitorare ordini (Scenario C), né associare correttamente indirizzi e recapiti di consegna (Scenario D).

Scenario (B) - Gestione del Ristorante:

Il secondo macro-scenario (Gestione del Ristorante) copre la gestione completa del locale, dalle informazioni anagrafiche principali (nome, sede/luogo, contatti, identificativi fiscali - Partita IVA) fino alla costruzione del menù in vendita al pubblico.

In pratica, il ristorante deve poter:

- Autenticarsi nell'applicazione con il proprio account e operare solo sulle aree autorizzate;
- Aggiornare dati e preferenze del proprio profilo/ristorante;
- Cancellarsi quando previsto dalle regole applicative;
- Mantenere aggiornate le informazioni chiave del locale, tra cui **nome ristorante**, **numero di telefono**, **partita IVA**, **indirizzo** e metadati collegati.

*Dal punto di vista del catalogo, una volta registrato il ristorante può inserire i prodotti in vendita sia selezionandoli da una base comune (coerente con il dataset condiviso **documents/meals.json**), sia creando piatti personalizzati gestiti direttamente da lui.*

Per ciascun piatto devono essere gestite informazioni complete e coerenti, in particolare:

- **nome del prodotto;**
- **tipologia/categoria;**
- **prezzo;**
- **ingredienti/composizione;**
- **foto illustrativa.**

*Questa logica è riflessa nelle pagine dedicate al ristorante (**creaRistorante.html**, **gestioneRistorante.html**, **gestioneMenù.html**, **piattiGenerici.html**, **piattoPersonalizzato.html**, **modificaPiattoPersonalizzato.html**) e nel dominio backend dei ristoranti/piatti (**routes/restaurants.js**, **routes/meals.js**).*

“Relazione con gli altri Scenari”: la qualità e completezza di questo scenario determina direttamente ciò che il cliente vede e acquista (Scenario C), e influenza anche fattibilità e correttezza delle consegne (Scenario D), oltre a dipendere dai controlli identitari/ruolo del profilo utente (Scenario A).

Scenario (C) - Gestione degli Ordini:

Il terzo macro-scenario (Gestione degli Ordini) riguarda la classica gestione delle attività del cliente in un sistema di ordinazione online, dalla preparazione dell'account fino alla chiusura dell'acquisto.

Dal punto di vista dell'utente, il flusso include:

- Registrazione, login, aggiornamento dati/preferenze ed eventuale cancellazione del profilo;
- Memorizzazione delle informazioni personali (es. nome, cognome) e dei dati dell'account;
- Associazione delle informazioni di pagamento (es. carta di credito o carta prepagata);
- Selezione di preferenze utili alla personalizzazione del servizio (es. offerte speciali in base ai prodotti preferiti).

Dopo l'accesso, l'utente registrato può consultare il menù, selezionare uno o più piatti, aggiungerli al carrello, rivedere il contenuto e concludere l'acquisto; l'ordine passa quindi alla fase operativa che porta alla consegna.

Nel progetto questo comportamento è distribuito tra Frontend e Backend:

- **Lato Cliente:** consultazione menù, composizione carrello (**carrello.html**), conferma ordine e storico (**cliente/ordini.html**);
- **Lato Backend:** organizzazione del carrello e della creazione ordine tramite **routes/carts.js** e **routes/orders.js**;
- **Lato Ristoratore:** visualizzazione e avanzamento dello stato ordine (**ristoratore/ordini.html**).

“Relazione con gli altri Scenari”: gli ordini dipendono dal profilo autenticato e dalle preferenze utente (Scenario A), dall’offerta prodotti configurata dal ristorante (Scenario B) e generano i dati logistici necessari alla consegna (Scenario D).

Scenario (D) - Gestione delle Consegne:

Quest’ultimo macro-scenario governa la parte finale del processo ordine, includendo stati di avanzamento, modalità di ritiro/consegna e dati logistici necessari alla chiusura corretta della richiesta.

Il flusso di stato di un ordine è descritto come sequenza operativa:

1. **ordinato;**
2. **in preparazione;**
3. **in consegna;**
4. **consegnato.**

Il progetto considera due modalità di ritiro:

- **Ritiro presso il Ristorante:**
 - *Viene fornito un tempo di attesa stimato, calcolato considerando la coda di ordini in preparazione;*
 - *Quando il ristorante segnala l’ordine come pronto/preparato, l’ordine esce dalla coda di lavorazione del ristorante e passa allo stato finale di consegnato (ritiro completato).*
- **Consegna a Domicilio:**
 - *In fase d’ordine, il cliente richiede la consegna a domicilio e specifica l’indirizzo/punto di recapito;*
 - *Viene calcolato un **costo di consegna** dipendente dalla distanza tra ristorante e luogo di consegna;*
 - *Il calcolo distanza può essere supportato dalle API cartografiche (es. OpenStreetMap), così da derivare i chilometri e determinare il costo logistico;*
 - *Quando l’ordine arriva a destinazione, **l’utente finale conferma la ricezione**, dopodichè questa segnalazione chiude il ciclo logistico e fa transitare l’ordine dallo stato **in consegna** allo stato **consegnato**.*

*Nel progetto, questa logica è supportata da componenti già presenti: stato ordine e avanzamento lato **routes/orders.js**, dati carrello/checkout in **routes/carts.js** e validazione/coerenza indirizzi in **utils/addressValidation.js**.*

“Relazione con gli altri Scenari”: la consegna chiude il flusso end-to-end iniziato con autenticazione utente (Scenario A), alimentato dal catalogo ristorante (Scenario B) e formalizzato nell’ordine (Scenario C), trasformando il dato digitale dell’ordine in servizio effettivamente erogato.

2. Struttura generale del Progetto:

2.0 Entry-point e Configurazione:

- **index.js:** avvio del server, inizializzazione middleware e montaggio delle route API.
- **package.json** e **package-lock.json:** dipendenze, script npm, configurazione lock delle librerie.

2.1 API Backend:

- Cartella *routes/* con i moduli endpoint:
 - **users.js**
 - **restaurants.js**
 - **meals.js**
 - **carts.js**
 - **orders.js**

Questi file rappresentano il layer HTTP e incapsulano le operazioni CRUD → (Create, Read, Update, Delete - Creazione, Lettura, Aggiornamento, Eliminazione) e le business rules per ciascun dominio.

2.2 Middleware di Sicurezza/Autorizzazione:

- **middlewares/authenticateUser.js**: verifica dell'identità utente (tipicamente token/sessione) e iniezione del contesto utente nella request.
- **middlewares/authorizeRistoratore.js**: enforcement del controllo di ruolo per proteggere endpoint dedicati ai ristoratori.

2.3 Utility e Configurazioni:

- **utils/config.js**: variabili e costanti di configurazione condivise.
- **utils/addressValidation.js**: logica di validazione indirizzi (coerenza input geografico/consegna).

2.4 Asset documentali e Dati di supporto:

- **documents/swagger.json + documents/swagger.js**: specifica OpenAPI e setup della documentazione endpoint.
- **documents/meals.json**: dataset per i piatti.
- **documents/relazione_tecnica.md**: documento tecnico di riferimento.
- **documents/PWM__project_25_26.pdf**: specifica/brief progettuale originale.

2.5 Frontend web statico:

- Cartella *public/* con pagine HTML per autenticazione, profilo e dashboard.
- Sottocartelle per ruolo:
 - **public/cliente/** (home, menù, carrello, ordini, offerte)
 - **public/ristoratore/** (creazione ristorante, gestione menù, ordini, statistiche, bacheca, piatti)
- Asset grafici e stile in **public/assets/**:
 - CSS3 theme (**modern-theme.css**) e responsive (**responsive.css**)
 - Script auth client-side (**auth.js**)
 - Immagini branding (**logo.png**, **Favicon.png**)

3. Scelte Architetture e Implementative:

3.0 Organizzazione modulare del Backend:

L'uso di route separate per dominio (*users, restaurants, meals, carts, orders*) permette:

- **manutenibilità**: ogni file tratta un sottoinsieme funzionale coerente;
- **estendibilità**: nuove funzionalità si aggiungono senza impatto trasversale elevato;
- **testabilità**: è più semplice validare endpoint per contesto.

3.1 Sicurezza a livelli:

La sicurezza è strutturata su più strati:

1. **Autenticazione** via middleware dedicato (**authenticateUser.js**).
2. **Autorizzazione per ruolo** (**authorizeRistoratore.js**) sui percorsi riservati.
3. **Validazione input** (include utility come **addressValidation.js**) per ridurre dati inconsistenti.

Questa impostazione evita che un utente non autorizzato possa accedere ad aree di gestione ristorante.

3.2 Frontend segmentato per ruolo:

La suddivisione in pagine separate sotto **public/cliente/** e **public/ristoratore/** è una scelta funzionale:

- Semplifica il flusso utente;
- Riduce la complessità di ogni vista;
- Riflette direttamente i casi d'uso della piattaforma.

3.3 Design di Sistema:

Con **modern-theme.css** e **responsive.css** si ottiene:

- Coerenza visiva tra pagine;
- Adattabilità a dispositivi diversi (desktop/tablet/mobile);
- Separazione netta tra contenuto (HTML) e presentazione (CSS).

3.4 Vincoli Backend, Persistenza e Paradigma REST:

Il Progetto adotta un Backend realizzato con **Node.js** e organizzato per moduli di dominio; il vincolo di archiviazione dei dati è coerente con un'impostazione orientata a **MongoDB** per la memorizzazione dei dati applicativi (utenti, ristoranti, menù, ordini e informazioni correlate).

A livello di integrazione applicativa, le informazioni mostrate nelle pagine web non sono definite direttamente nel codice dell'interfaccia, ma devono essere recuperate e aggiornate tramite chiamate al Backend secondo Paradigma **REST**. Questo implica che, per ogni risorsa principale, siano previsti endpoint per:

- **presentazione/lettura** delle informazioni (operazioni di consultazione);
- **modifica/aggiornamento** delle informazioni (operazioni di editing);
- eventuale creazione/rimozione dove previsto dal dominio funzionale.

Nel Progetto, tale impostazione è rappresentata dalla separazione in route (**routes/users.js**, **routes/restaurants.js**, **routes/meals.js**, **routes/carts.js**, **routes/orders.js**) che fungono da strato API per il Frontend.

È inoltre previsto e fornito il requisito di documentazione delle API tramite **Swagger/OpenAPI**, con file **documents/swagger.json** e setup **documents/swagger.js**, così da rendere espliciti contratti, endpoint e payload necessari all'integrazione Frontend-Backend.

4. Presentazione del sito web (UX/UI):

4.0 Area Cliente:

- **home.html**: panoramica iniziale e accesso rapido ai contenuti.
- **menù.html**: consultazione catalogo piatti.
- **carrello.html**: revisione articoli e preparazione checkout.
- **ordini.html**: storico e stato ordini.
- **offerte.html**: promozioni e contenuti commerciali.

Il flusso cliente segue la seguente sequenza: **esplora** → **seleziona** → **acquista** → **monitora**.

4.1 Area Ristoratore:

- Setup e identità del locale: **creaRistorante.html**, **gestioneRistorante.html**
- Gestione prodotto/catalogo: **gestioneMenù.html**, **piattiGenerici.html**, **piattoPersonalizzato.html**, **modificaPiattoPersonalizzato.html**
- Gestione operativa: **ordini.html**, **statistiche.html**, **gestioneBacheca.html**, **home.html**

Il flusso ristoratore supporta sia fase iniziale (configurazione) sia operatività giornaliera (ordini, aggiornamenti, analisi).

5. Operazioni Base richieste in fase di discussione progetto e come sono state realizzate:

- **Registrazione e login al sito**: supportate dal flusso account (pagine **register.html**, **login.html**, **logout.html**) e dalla gestione utenti lato API (**routes/users.js**).
- **Visualizzazione informazioni su piatti, clienti, ristoranti e acquisti**: i piatti e i dati di catalogo sono mostrati nelle interfacce utenti cliente/ristoratore e gestiti via **routes/meals.js**; le informazioni di account e storico acquisti sono disponibili nelle aree profilo/ordini (**profilo.html**, **cliente/ordini.html**) con supporto Backend **routes/users.js** e **routes/orders.js**.
- **Ricerca ristoranti** (per luogo e nome): implementabile attraverso i flussi di consultazione ristoranti (Frontend cliente) e i filtri/endpoint del dominio **routes/restaurants.js**.
- **Ricerca piatti** (per tipologia, nome, prezzo): gestita nel dominio menù/piatti (**routes/meals.js**) e nelle pagine di esplorazione (**cliente/menù.html**, pagine gestione menù ristoratore).
- **Login e ordine di uno o più piatti**: percorso completo cliente autenticato → scelta piatti → carrello (**cliente/carrello.html**) → conferma ordine (**routes/carts.js**, **routes/orders.js**).
- **Gestione consegne**: coperta da stato ordine, modalità ritiro/consegna e validazione indirizzo (**routes/orders.js**, **utils/addressValidation.js**).
- **Visualizzazione statistiche per ristorante**: prevista nell'area ristoratore con pagina dedicata **ristoratore/statistiche.html** e dati aggregati ricavabili dal dominio ordini.
- **Visualizzazione acquisti presenti e passati per cliente**: disponibile nello storico ordini del cliente (**cliente/ordini.html**) con supporto delle route ordini.

Queste operazioni sono relazionate tra loro in un unico flusso: l'utente si autentica, ricerca ristoranti/piatti, effettua acquisti, monitora ordini e consegne, mentre il ristoratore controlla offerta, ordini e statistiche.

5.0 Operazioni e vincoli tecnici:

Oltre alle operazioni base, il progetto copre e inquadra le operazioni e vincoli richiesti in discussione e organizzazione progettuale.

- **Ricerca del ristorante per piatto:** la relazione tra catalogo piatti e ristorante consente di risalire ai locali che vendono un determinato prodotto, combinando dati di dominio ristoranti (*routes/restaurants.js*) e piatti (*routes/meals.js*).
- **Ricerca di piatti per ingredienti:** il modello dei piatti include la composizione/ingredienti, dunque è possibile filtrare il menù per ingrediente, sia in ottica consultazione cliente sia in gestione catalogo ristorante.
- **Ricerca di piatti per allergie:** partendo dai metadati degli ingredienti è possibile impostare filtri di esclusione/preferenze alimentari (facoltative), così da mostrare piatti compatibili con i vincoli alimentari dell'utente.

Inoltre, il progetto prevede il requisito di bootstrap: **allo startup applicativo**, i dati iniziali necessari devono essere disponibili nel Backend. Questo vincolo è coerente con la presenza del dataset JSON (*documents/meals.json*) e con la fase di setup/caricamento della documentazione.

5.1 Gestione Utenti e Autenticazione:

- **Registrazione/login/logout:** pagine dedicate nel Frontend e route *users.js* nel Backend.
- **Protezione risorse:** middleware di autenticazione applicato alle routes sensibili.

“Scelta Implementativa”: mantenere separati endpoint utente e controlli di sicurezza per un miglior uso e maggiore chiarezza.

5.2 Gestione Ristoranti:

- Route *restaurants.js* per creazione, aggiornamento e consultazione ristoranti.
- Dashboard ristorante con schermate dedicate alla gestione dello stato del locale.

“Scelta Implementativa”: accoppiare strettamente il dominio “ristorante” al ruolo ristorante, con restrizioni lato middleware.

5.3 Gestione Menù e Piatti:

- Route *meals.js* per CRUD su prodotti/piatti.
- Supporto a piatti generici e personalizzati tramite pagine differenziate.
- Dataset *documents/meals.json* come base dati iniziale/di esempio.

“Scelta Implementativa”: isolare i dati (JSON/documenti) dalla UI per facilitare l'estensione futura verso un database completo.

5.4 Carrello e Ordini:

- **cars.js**: logica di composizione carrello lato backend.
- **orders.js**: creazione e monitoraggio ordini.
- Pagine cliente dedicate (**carrello.html**, **ordini.html**) e pagina ristoratore ordini (**public/ristoratore/ordini.html**).

“Scelta Implementativa”: suddividere i due domini, pur collegati, per mantenere la semantica delle API più pulita.

5.5 Validazione Indirizzi e coerenza Consegna:

- Utilizza **addressValidation.js** per validare l’input di recapito/consegna.

“Scelta Implementativa”: estrarre la logica di validazione in funzioni comuni, evitando la duplicazione nei controller delle routes.

5.6 Documentazione API:

- **swagger.json** come documentazione API formalizzata.
- **swagger.js** per esposizione/integrazione documentazione.

“Scelta Implementativa”: includere la documentazione nel progetto per allineare sviluppo Backend e Frontend.

6. Operazioni e funzionalità aggiuntive implementate a piacere:

6.0 Esperienza multi-ruolo (navigazione intelligente):

- Gestione avanzata della sessione lato client con controllo coerenza token/ruolo, supporto a token separati per ruolo e sincronizzazione stato (**public/assets/auth.js**).
- Salvataggio e ripristino dell’ultima pagina visitata dopo autenticazione, con fallback differenziati per cliente e ristoratore.
- Controlli preventivi per ridurre errori di navigazione tra aree con permessi diversi.

Valore aggiunto: riduce possibili conflitti durante l’uso quotidiano, rende più stabile l’accesso alle aree riservate e migliora la UX percepita su refresh/re-login.

6.1 Personalizzazione offerte e bacheca dinamica:

- Area ristoratore con strumenti dedicati alla costruzione della **bacheca offerte**, con gestione selettiva dei piatti e aggiornamento incrementale.
- Area cliente con consumo delle offerte e filtraggio in base alle preferenze alimentari facoltative, così da proporre contenuti più pertinenti.
- Supporto a scenari multipli di comunicazione commerciale (offerte presenti, assenza preferenze, offerte non compatibili).

6.2 Qualità del checkout di alto livello:

- *In checkout cliente è presente una validazione approfondita dell'indirizzo con logica di parsing e controlli semantici.*
- *La validazione non si limita alla sola presenza del campo, ma verifica coerenza dei componenti utili alla consegna.*

Valore aggiunto: *diminuisce errori logistici, ordini non consegnabili e problematiche verificatesi in post-acquisto.*

6.3 Gestione operativa degli ordini più completa (stati, conferme e notifiche):

- *Distinzione chiara tra ordini attivi e storico lato ristorante.*
- *Transizioni stato guidate in interfaccia, con gestione del ciclo fino alla consegna.*
- *Conferma di ricezione da parte del cliente e meccanismi di lettura/gestione notifiche nel ciclo di consegna.*

Valore aggiunto: *maggiore tracciabilità end-to-end e riduzione delle ambiguità tra “preparato”, “in consegna” e “consegnato”.*

6.4 Dashboard Statistiche:

- *Presenza di una vista dedicata alle statistiche ristorante (**ristoratore/statistiche.html**) con (KPI – Key Performance Indicators) e aggregazioni.*
- *Organizzazione dei dati in forma leggibile per monitoraggio andamento vendite/ordini.*

Valore aggiunto: *il sistema non è solo transazionale, ma anche decisionale; supporta il ristorante nell'ottimizzazione del business.*

6.5 Robustezza complessiva lato progettuale (oltre il CRUD basilare):

- *Struttura Backend modulare per domini + sicurezza a livelli (autenticazione, autorizzazione ruolo, validazione input).*
- *Documentazione API potenziata (Swagger/OpenAPI), utile anche in ottica di manutenzione, test e passaggio di consegne progettuale.*
- *Coerenza UI cross-device con tema condiviso e regole responsive dedicate.*